The Stata Journal (*yyyy*) *vv*, Number *ii*, pp. 1–21

disco: Distributional Synthetic Controls

Florian Gunsilius
Emory University
Atlanta, Georgia/US
fgunsil@emory.edu

David Van Dijcke
University of Virginia
Charlottesville, Virginia/US
eju5az@virginia.edu

Abstract. The method of synthetic controls is widely used for evaluating causal effects of policy changes in settings with observational data. Often, researchers aim to estimate the causal impact of policy interventions on a treated unit at an aggregate level while also possessing data at a finer granularity. In this article, we introduce the new `disco` command, which implements the Distributional Synthetic Controls method introduced in Gunsilius (2023, *Econometrica* 91: 1105–1117). This command allows researchers to construct entire synthetic distributions for the treated unit based on an optimally weighted average of the distributions of the control units. Several aggregation schemes are provided to facilitate clear reporting of the distributional effects of the treatment. The package offers both quantile-based and cumulative distribution function–based approaches, comprehensive inference procedures via bootstrap and permutation methods, and visualization capabilities. We empirically illustrate the use of the package by replicating the results in Van Dijcke et al. (2026, *Review of Economics and Statistics*, forthcoming).

Keywords: st0001, disco, disco_estat, disco_plot, disco_weight, synthetic control, treatment effect, distributional analysis, causal inference, optimal transport

1 Introduction

Synthetic control methods, pioneered by Abadie and Gardeazabal (2003) and formalized in Abadie et al. (2010), have revolutionized policy evaluation with aggregate data. These methods have become a cornerstone of modern causal inference, particularly in settings where traditional regression approaches are infeasible due to a limited number of treated units or the lack of a good counterfactual control group. While these methods have proven invaluable for estimating average treatment effects, they traditionally focus on matching means, potentially overlooking rich heterogeneous effects across the distribution of outcomes that are often of primary interest to researchers and policymakers.

The Distributional Synthetic Controls (DiSCo) method (Gunsilius 2023) addresses this limitation by extending the synthetic control framework to match entire outcome distributions rather than just means. This extension is particularly valuable in empirical settings where researchers have access to repeated cross-sectional data within aggregate units but cannot track individual units over time. Such scenarios are common in policy evaluation, where interventions occur at an aggregate level (e.g., states, counties, or firms) but researchers have access to individual-level or more granular data within these units. In these contexts, treatment effects may vary substantially across the distribution of outcomes, making that distribution a crucial object of study.

This article introduces the `disco` command in Stata, which implements the Distributional Synthetic Controls method. We first provide a practical overview of the underlying methodology, then lay out the command's syntax and usage, and illustrate its use in an empirical application. A companion R package is available at Van Dijcke et al. (2024).

2 Methodological Background

2.1 Overview and Notation

Consider a panel of $J+1$ units (for example, firms) observed over T periods. One unit (conventionally labeled as unit 1) receives treatment after period T_0 (or equivalently in period $t_0 = T_0 + 1$), while the remaining J units serve as potential controls. For each unit j and time period t, we observe individual-level outcomes Y_{ijt} (for example, employee wages). From these, we can construct the cumulative distribution function (CDF) $F_{Y_{jt}}$

$$F_{Y_{jt}}(y) = P(Y_{jt} \leq y)$$

which gives the probability P that the outcome of unit j at time t, Y_{jt}, is less than or equal to y, and its corresponding quantile function $F^{-1}_{Y_{jt}}$,

$$F^{-1}_{Y_{jt}}(q) = \inf\{x \in \mathbb{R} : q \leq F(x)\} \tag{1}$$

which captures the value in the support of Y_{jt} that is larger than $q \times 100$ percent of the distribution. These functions capture the entire distribution of outcomes within each aggregate unit, preserving information about heterogeneity that would be lost by focusing only on means.

The goal is to estimate what the distribution of outcomes in the treated unit would have been in the absence of treatment. We denote this counterfactual distribution's CDF as $F_{Y_{1t,N}}$ and its quantile function as $F^{-1}_{Y_{1t,N}}$ for periods $t > T_0$. The DiSCo method constructs entire synthetic distributions for the treated unit using optimally chosen weights on the control distributions.

2.2 Quantile-Based Approach

The quantile-based approach constructs the counterfactual by finding optimal weights λ_j that create a synthetic control distribution via a weighted average of control unit quantile functions:

$$F^{-1}_{Y_{1t,N}}(q) \;=\; \sum_{j=2}^{J+1} \lambda_j^* F^{-1}_{Y_{jt}}(q) \quad \text{for } q \in (0,1) \tag{2}$$

These weights are chosen to minimize the squared distance between the treated unit's pre-treatment quantile function and the weighted combination of control unit quantile

functions, corresponding to minimizing the 2-Wasserstein distance,

$$\boldsymbol{\lambda}_t^* \;=\; \text{argmin}_{\lambda \in \Delta^J} \int_{\text{qmin}}^{\text{qmax}} \left| \sum_{j=2}^{J+1} \lambda_j F_{Y_{jt}}^{-1}(q) \;-\; F_{Y_{1t}}^{-1}(q) \right| dq \tag{3}$$

for $t \leq T_0$, where $(\boldsymbol{\lambda}_t^*) = (\lambda_{1t}^*, \ldots, \lambda_{jt}^*)$ is the vector of J weights (one for each control unit), and Δ^J is the J-dimensional unit simplex, which constrains the weights to satisfy $\sum_{j=2}^{J+1} \lambda_j = 1$ and $\lambda_j \geq 0$ for all $j > 1$. Then, the overall synthetic control weights are computed as a simple average over the pre-treatment weights,

$$\lambda_j^* = \frac{1}{T_0} \sum_{t=1}^{T_0} \lambda_{jt}^*$$

Here, qmin and qmax default to 0 and 1, respectively, matching the entire distribution. In practice, researchers may sometimes wish to match or conduct inference on specific parts of the distribution. To accommodate this, the command allows users to specify the `qmin` and `qmax` options, restricting the integral to an interval $[a, b] \subset [0, 1]$.

To relax the positivity constraint and allow for extrapolation as in Doudchenko and Imbens (2016), the user can specify the `nosimplex` option in the main command. The primary motivation for choosing this particular objective function is that it has the intuitive interpretation of a "regression of quantile functions." Alternatively, one can interpret the problem as finding the set of weights that makes the treated unit's quantile function as similar as possible to the weighted average of the control units' quantile functions (Gunsilius 2023, p. 1109). This aligns with the interpretation of the weighted average quantile function, $\sum_{j=2}^{J+1} \lambda_j^* F_{Y_{jt}}^{-1}(q)$, as a Wasserstein barycenter (Agueh and Carlier 2011).

Note that problem (3) is "distributional" in the sense that it assigns a single weight to each quantile function in its entirety, which can be seen from the outer integration over y. Thus, the synthetic control is truly a weighted average of distributions, rather than a pointwise average of quantiles as in (Chen 2020). The benefits of working with full distributions as the fundamental objects of estimation are: (1) it corresponds to the classical synthetic controls approach of assigning one single weight to every unit j; and (2) it allows for individuals to drop in and out of a unit, meaning one need not track individuals over time. This accommodates settings with entry and exit (e.g., employees leaving firms, as in Van Dijcke et al. (2026)), and settings where the researcher may only have a repeated cross-section rather than a balanced panel of observations.

In practice, the integral in (3) must be simulated, resulting in the empirical problem,

$$\boldsymbol{\lambda}_i^* = \underset{\bar{\lambda} \in \Delta^J}{\text{argmin}} \frac{1}{G} \sum_{g=1}^{G} \sum_{j=2}^{J+1} \left| \lambda_{jt} F_{Y_{jt}^{-1}}^{-1}(V_g) - F_{Y_{1t}}^{-1}(V_g) \right|^2 \tag{4}$$

where V_g are G independent draws from the uniform distribution on $[0, 1]$, with G large. In practice, the command simply uses uniformly spaced draws from $[0, 1]$. We also

use a single parameter G as both the number of simulation draws for the integral and the number of grid points on which we evaluate the synthetic control distributions in practice.

2.3 CDF-Based Approach

As an alternative, one can match the cumulative distribution functions directly, minimizing the 1-Wasserstein distance:

$$\boldsymbol{\lambda}_t^* \;=\; \text{argmin}_{\lambda \in \Delta^J} \int_{-\infty}^{\infty} \left| \sum_{j=2}^{J+1} \lambda_j F_{Y_{jt}}(y) \;-\; F_{Y_{1t}}(y) \right| dy \tag{5}$$

The synthetic control distribution function is then estimated analogously to (2) as

$$F_{Y_{1t,N}}(y) \;=\; \sum_{j=2}^{J+1} \lambda_j^* F_{Y_{jt}}(y) \quad \text{for } y \in \text{supp}(Y)$$

with $\text{supp}(Y)$ the support of the outcome variable Y. Of course, one can easily obtain a synthetic control quantile function from this by using the pseudo-inverse in (1), and vice versa for (2). Indeed, the command always computes both synthetic quantile functions and CDFs, regardless of whether the `mixture` option is specified, and stores them in `e(quantile_synth)` and `e(cdf_synth)`.

The CDF-based approach is often advantageous for discrete outcomes or settings where preserving specific portions of the distribution is critical (Gunsilius 2023, §4.3). To have the command solve (5) instead of (3), one can specify the `mixture` option. A prominent example where the mixture (CDF-based) approach is useful is when the outcome is a categorical variable. In that case, a weighted average of quantile functions, as computed in (3), will interpolate between the support points, which is undesirable. A "mixture" of distribution functions, however, restricts the weighted average to the same support points as the "donor" distribution functions (see Figure 1 in Gunsilius (2023)). We further illustrate this with a simple simulation in Stata: our target (treated) distribution is a discrete uniform distribution with equal probability mass on $\{1, 2, 3, 4\}$, i.e.,

$$Pr(Y_t = i) = \frac{1}{4} \text{ for } i = 1, \ldots, 4$$

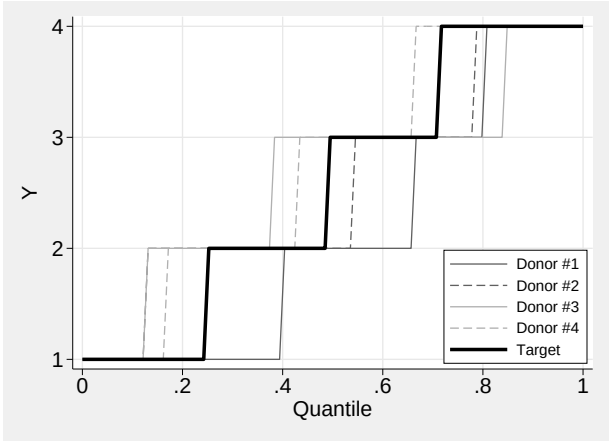
Our donor distributions take this target distribution but place more mass at one of the support points each, i.e.,

$$Pr(Y_j = i) = \begin{cases} 1/5 & \text{if } i \neq j, \\ 2/5 & \text{if } i = j, \end{cases}$$

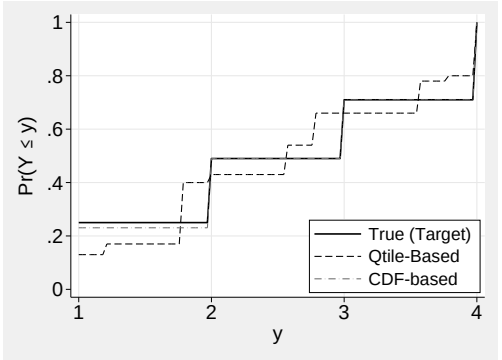
for $i, j = 1, \ldots, 4$. The "raw" control and target quantile functions corresponding to these random variables are plotted together in Figure 1a. The synthetic CDFs and quantile functions, constructed by either solving (3) (dashed) or (5) (dash-dot), are

plotted against the target distribution (solid) in panels 1b and 1c. As clearly shown, the CDF-based synthetic controls correctly preserve the support of the control (donor) functions, while the quantile-based approach undesirably interpolates between support points. Of course, such interpolation *would* be desirable for a continuous outcome variable.

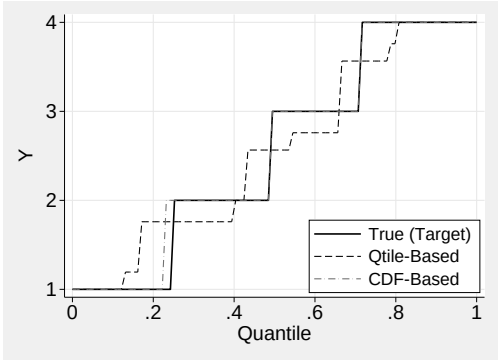
Finally, note that when the outcome variable is categorical, one should specify the `g` and `m` options to reflect the number of support points. For instance, if the variable takes values on the integers from 1 to 10, one would specify `g(10)` and `m(10)`. This ensures that the integral in (5) correctly sums over the support points. The code expects evenly spaced support points for this to work. If the outcome variable is not evenly spaced, one can normalize it without affecting the estimates. Furthermore, when creating graphs for categorical variables with `disco_plot`, one may want to specify the `categorical` option to change the graph type to a bar plot instead of a line plot.



(a) Target and Control Quantile Functions



(b) Target and Synthetic CDFs



(c) Target and Synthetic Quantile Functions

Figure 1: Illustrative Simulation of `mixture` Option

2.4 Inference

Permutation Test

When the `permutation` option is specified, the command carries out the distributional version of the classical permutation test for synthetic controls proposed in Gunsilius (2023) and further developed in Van Dijcke et al. (2026). The idea is intuitive: recompute the DiSCo estimator anew for each of the J control units, while pretending that the given control unit was, in fact, treated. If the treatment effect on the treated unit is not driven by statistical noise, the estimated treatment effect on the true treated unit should be an extreme value of the resulting distribution of "placebo" treatment effects.

The only complication in the distributional setting is that each single permutation delivers an entire distribution of treatment effects, so we need a way to aggregate these into a single statistic that we can rank. Echoing the use of the root mean squared prediction error in Abadie et al. (2010), Gunsilius (2023) proposed using the 2-Wasserstein distance. Then, an analogous permutation test to Abadie et al. (2010) is constructed by normalizing the average post-treatment prediction error by the average pre-treatment one (Van Dijcke et al. 2026, p.10),

$$r_j = \frac{R_j\left(T^* + 1, T\right)}{R_j\left(1, T^*\right)}$$

where

$$R_j\left(t_1, t_2\right) = \left(\frac{1}{t_2 - t_1 + 1} \sum_{t=t_1}^{t_2} d_t^2\right)^{1/2}$$

the root mean squared prediction error in the 2-Wasserstein distance at time t,

$$d_t^2 = \int_{q_{\min}}^{q_{\max}} \left| \sum_{j=1}^{J} \lambda_j^* F_{jt}^{-1}(q) - F_{0t}^{-1}(q) \right|^2 \mathrm{d}q$$

with λ_j^* the optimal DiSCo weights. As explained above, one can test for effects in a restricted part of the distribution by setting `qmin` > 0 or `qmax` < 1. Then we calculate the p-value of the permutation test as

$$p = \frac{1}{J+1} \sum_{i=0}^{J} 1\left\{r_j \geq r_0\right\}$$

which calculates how frequently the normalized placebo effects exceed the true effect on the treated. The null hypothesis that the effect on the treated is drawn from the placebo distribution can be rejected at standard significance levels, for instance $p \leq 0.05$. This p-value is stored in the `e(pval)` scalar by the `disco` command when the `permutation` option is specified.

Bootstrapped Confidence Intervals

The permutation test described above provides an intuitive, finite-sample approach to inference that has become standard in synthetic controls. In addition to the permutation test, we also provide a bootstrap procedure to compute confidence intervals that account for the randomness in the estimation of the distributions. It can be requested by specifying the `ci` option. Note that this can be computationally intensive, depending on the number of bootstrap replications requested via the `boots` option. If `ci` is specified, the `disco_plot` and `disco_estat` commands will automatically include confidence intervals and/or bootstrapped standard errors.

In Van Dijcke et al. (2026, Theorem 1), we prove that the bootstrap is uniformly valid for the DiSCo method up to a negligible set, meaning that the bootstrapped empirical process,

$$\tilde{\mathbb{G}}_n = \sqrt{n}\left(\tilde{F}^{-1}_{0tn,N} - \hat{F}^{-1}_{0tn,N}\right) = \sqrt{n}\left(\sum_{j=1}^{J} \tilde{\lambda}^*_{jn}\tilde{F}^{-1}_{jtn} - \sum_{j=1}^{J} \hat{\lambda}^*_{jn}\hat{F}^{-1}_{jtn}\right) \tag{6}$$

converges uniformly to the "true" empirical process,

$$\mathbb{G}_n = \sqrt{n}\left(\hat{F}^{-1}_{0tn,N} - F^{-1}_{0tn}\right) = \sqrt{n}\left(\sum_{j=1}^{J} \hat{\lambda}^*_{jn}\hat{F}^{-1}_{jtn} - \sum_{j=1}^{J} \lambda^*_{j}F^{-1}_{jtn}\right)$$

where $\tilde{\lambda}_{jn}$ and $\tilde{F}^{-1}_{jtn}$ are the bootstrapped optimal weights and quantile functions.

In practice, the following pseudo-code provides a high-level overview of how the bootstrapped confidence intervals are computed. For a more detailed exposition, see Algorithm 1 in Van Dijcke et al. (2026).

1. Estimate the main DiSCo weights λ^*_j on the original pre-treatment data by solving (the empirical version of) (3) or (5).
2. Construct the main post-treatment estimate — the synthetic quantile function in (2).
3. For b from 1 to `boots`:
   a. Resample each firm's data (pre + post) to create a bootstrap sample.
   b. Re-estimate weights on the bootstrapped pre-treatment data.
   c. Construct the estimate for each bootstrapped post-treatment $t > T_0$, using (1) the new weights calculated with the resampled pre-treatment quantile functions and (2) the resampled post-treatment quantile functions: $\sum_{j=1}^{J} \tilde{\lambda}_j \tilde{F}^{-1}_{jt}$.
   d. Compute the bootstrap "gap" in (6).
4. Form confidence intervals/bands using the empirical distribution of the bootstrap "gaps."

2.5 Relation to Classical Synthetic Controls

The DiSCo method reduces to the classical synthetic controls method of Abadie and Gardeazabal (2003) when only aggregate data are given — i.e., when each unit j has a

single aggregate observation rather than an entire distribution of observations (Gunsilius 2023, §4.1). As such, it complements the classical method for settings where researchers have access to sub-aggregate data *within* each unit j and want to (1) use all available data to inform estimation rather than collapsing it down to a single point for each unit and/or (2) estimate distributional treatment effects.

Several synthetic control commands already exist in Stata. The `synth` command implements the canonical estimator of Abadie and Gardeazabal (2003), which constructs a synthetic control for *scalar-valued* outcomes by constructing a convex weighted average of control units' outcomes, where each unit has a single number as its outcome value in each period. `gsynth` (Xu 2017, now absorbed into the `fect` package) builds on top of that by explicitly estimating the control units' outcomes through interactive fixed-effects models, which lets it handle several treated units and staggered timing while relaxing the parallel-paths assumption. Nonparametric variants such as `npsynth` loosen the functional form of the fit, but they too model settings where the outcome is a single number per period for each unit.

`disco` differs in the object it matches. The unit of estimation is the whole quantile function, or the whole CDF, so the command puts one weight on each control distribution and returns a counterfactual distribution for every post-treatment period instead of a single number. The most common setting where an entire distribution is observed for each unit in each period is the grouped data setting, building on Hausman and Taylor (1981) – see also Chetverikov et al. (2016) for grouped quantile IV and Van Dijcke (2025) for an analogous setting in the regression discontinuity design. In that sense, the `disco` command is a direct extension of the `synth` command's estimation approach to a grouped data setting with distribution-valued outcomes. The two approaches coincide when each unit's distribution is degenerate or one only observes the average: in that case, the objective in (3) reduces to the squared distance between scalars, and the `disco` weights are exactly the `synth` weights when using only the pre-treatment outcomes as predictors. A researcher who only needs a mean effect, or who has one observation per unit, should use `synth` or `gsynth`. `disco` earns its extra cost when each unit holds many observations and the researcher is interested in distributional treatment effects: which part of the tenure distribution loses mass after a return-to-office mandate, for example.

2.6 Implementation Details

The command uses a C++ plugin for the constrained quadratic optimization in (3) to improve performance, based on the `quadprog` code by Di Gaspero (2016). This code implements the Goldfarb–Idani active-set dual method (Goldfarb and Idnani 1983). The C++ plugin is compatible with Linux/Unix, macOS (both Apple Silicon and Intel), and Windows system architectures. For the optimization in (5), the command relies on the `LinearProgram` class in Mata.

Memory requirements scale with the number of units (J), time periods (T), grid points (G), and bootstrap replications (B).

Runtime is dominated by the bootstrap. A point estimate is cheap, but each of

the `boots` replications repeats the entire weight estimation, so confidence intervals are where the time goes. On the tenure panel used below, with about one million employee-quarter observations across 32 firms and three quarters, a single quantile estimate runs in about a second; the same specification with `m(100)` and `ci boots(300)` takes about ten seconds on a MacBook Pro with an Apple M1 Pro chip and 16 GB of memory. Three options control the trade-off. `boots()` enters the runtime close to linearly and is the main expense once `ci` is on. `m()` fixes how many points approximate the integral in (4); it defaults to 1000, and lowering it speeds up every weight solve at some cost to the accuracy of that integral. `g()` sets how finely the quantile functions and CDFs are evaluated. In practice it pays to develop and debug with point estimates on a coarse grid, then run `ci` with the full `boots` count once, for the final numbers.

3 The `disco` Command

3.1 Syntax

`disco depvar idvar timevar` *if* [*in*] `, idtarget(`*varname*`) t0(#)` [*options*]

`depvar` is the outcome variable (numeric).

`idvar` is the unit identifier variable (numeric).

`timevar` is the time-period variable (numeric).

3.2 Options

Required Options

`idtarget(varname)` specifies the ID value of the treated unit in *idvar*.

`t0(#)` specifies the first time period of treatment in *timevar*. Note the slight difference in notation with Abadie and Gardeazabal (2003), who use capital T_0 to denote the last pre-treatment period.

Model Options

`m(#)` specifies the number of grid points for the approximation of the integral in (4). See the explanation there. The default is 1000.

`g(#)` specifies the number of grid points for the evaluation of the quantile functions and CDFs, i.e., for the vectors/functions that are returned by the command. The default is `g(100)`.

`mixture` requests the CDF-based approach using the 1-Wasserstein metric in (5) instead of the default quantile-based approach using the 2-Wasserstein metric in (3). This is recommended for categorical outcomes or when distributions are likely mixtures.

The mixture weights are found by solving a linear program with $2m + J$ variables, where m is set by `m()`; this becomes slow for large `m()`, so values up to around `m(100)` are recommended with this option.

`nosimplex` removes the constraint that weights must lie in the unit simplex, allowing for negative weights while maintaining that the weights sum to one. This enables extrapolation beyond the convex hull of control outcomes.

`qmin(#)` specifies the minimum quantile for the estimation range; the default is `qmin(0)`.

`qmax(#)` specifies the maximum quantile for the estimation range; the default is `qmax(1)`.

Inference Options

`ci` computes bootstrap confidence intervals for all distributional effects. Confidence intervals account for estimation uncertainty in both the weights and empirical distributions.

`boots(#)` specifies the number of bootstrap replications for confidence intervals. The default is `boots(300)`. More replications provide more stable intervals but increase computation time linearly.

`cl(#)` sets the confidence level as a percentage for intervals. The default is `cl(0.95)`. Must be between 0 and 1.

`permutation` performs a permutation test by applying the estimation to each control unit as if it were treated, following Abadie et al. (2010); Van Dijcke et al. (2026). Reports a p-value for the null of no effect.

`seed(#)` sets the random-number seed for reproducibility of bootstrap inference; the permutation test does not depend on the seed because the integral in (4) is evaluated on a fixed, uniformly spaced grid, so only the bootstrap uses random draws.[1]

`nouniform` requests pointwise rather than uniform confidence bands. Uniform bands control for multiple testing across quantiles but are wider than pointwise bands. Use of this option is discouraged since inference on functional objects should generally be uniform.

`agg` specifies the type of aggregation for summary statistics and plots. One of

- `"quantile"`: summarize estimated quantile functions
- `"cdf"`: summarize estimated CDFs
- `"quantileDiff"`: summarize differences in quantiles between treated and synthetic
- `"cdfDiff"`: summarize differences in CDFs between treated and synthetic

`samples(numlist)` specifies quantile or CDF points to aggregate over for summary statistics. For quantiles, these are in [0,1]. For CDFs, these are values of the outcome

1. This differs from the companion R package (Van Dijcke et al. 2024), which draws the quantile levels in (4) at random; there, the point estimates and the permutation test do depend on the seed.

variable. If not specified, the default is to partition the support (either [0,1] or the range of the outcome variable) into 5 equally spaced points ([0, 0.25, 0.5, 0.75, 1]) and aggregate the treatment effects within those intervals.

3.3 Stored Results

Scalars

`e(amin)`	minimum observed outcome value
`e(amax)`	maximum observed outcome value
`e(m)`	number of samples to approximate integrals in estimation step
`e(g)`	number of grid points to evaluate quantile function / CDF on
`e(t_max)`	last time period in the dataset
`e(N)`	number of observations
`e(pval)`	p-value from permutation test
`e(doci)`	indicates whether CIs were requested
`e(t0)`	first treatment period
`e(cl)`	confidence level (e.g., 0.95)

Macros

`e(cmd)`	`disco`
`e(agg)`	aggregation type (e.g., "quantile")
`e(cmdline)`	command as typed

Matrices

`e(cids)`	control IDs (1 × J)
`e(weights)`	synthetic control weights (J × 1)
`e(cdf_t)`	treated unit CDFs (G × T)
`e(cdf_synth)`	synthetic CDFs (G × T)
`e(quantile_t)`	treated unit quantiles (G × T)
`e(quantile_synth)`	synthetic quantiles (G × T)
`e(cdf_diff)`	CDF differences (G × T)
`e(quantile_diff)`	quantile differences (G × T)
`e(summary_stats)`	summary stats matrix (length of `samples` option - 1 × 7)

3.4 Post-estimation Commands

Three post-estimation commands are available after `disco`: `disco_estat` for summary tables of the aggregated treatment effects, `disco_plot` for visualizations, and `disco_weight` for inspecting the estimated weights.

The `disco_estat` command

The `disco_estat` command displays summary statistics after `disco` estimation. It provides a detailed summary of aggregated distributional treatment effects, including point estimates, standard errors, and confidence intervals across different time periods. Aggregation is done along the partition of the support specified in the `samples` option in the `disco` command.

Syntax `disco_estat summary`

Remarks `disco_estat` requires that `disco` has been run previously with a specific `agg` and `samples` option, which cannot be altered post-estimation. This limitation is imposed to avoid the command having to pass large bootstrap matrices back to Stata when the `ci` option is specified.

The `disco_plot` command

The `disco_plot` command creates visualizations after `disco` estimation. It displays quantile functions, CDFs, and their differences over time, with optional confidence intervals. Plot types are automatically adapted to the aggregation method used (e.g., `quantileDiff`, `cdfDiff`), and confidence intervals appear if `ci` was specified in `disco`.

Syntax `disco_plot` [, *options*]

Options The options for `disco_plot` are the following.

`agg(string)` specifies if the outcome variable is categorical and a CDF plot is requested in `agg`. If specified, a bar plot is created instead of a line plot.

`categorical` specifies if the outcome variable is categorical and a CDF plot is requested in `agg`. If specified, a bar plot is created instead of a line plot.

`title(string)` specifies a custom title for the graph. Defaults vary by plot type.

`ytitle(string)` specifies a custom y-axis title. Defaults vary by plot type.

`xtitle(string)` specifies a custom x-axis title. Defaults vary by plot type.

`color1(string)` specifies the color for the first series (default `"blue"`).

`color2(string)` specifies the color for the second series in level plots (default `"red"`).

`cicolor(string)` specifies the color for confidence intervals (default `"gs12"`).

`lwidth(string)` specifies the line width (default `"medium"`).

`lpattern(string)` specifies the line pattern for the second series (default `"dash"`).

`legend(string)` specifies legend options, passed directly to Stata's graph commands; see [R] **legend_options**.

`byopts(string)` specifies options for small multiples using `by()`, such as `rows(2)` or `ytitle`; see [R] **by_option**.

`plotregion(string)` specifies options for the plot region in Stata graphs; see [R] **region_options**.

`graphregion(string)` specifies options for the overall graph region; see [R] **region_options**.

`scheme(string)` specifies a scheme name for the graph; see [R] **scheme_option**.

`hline(real)` y coordinate for a dashed grey horizontal line; see [R] **added_line_options**.

`vline(real)` x coordinate for a dashed grey vertical line; see [R] **added_line_options**.

`xrange(numlist)` numlist of size 2 to set the x range; see [R] **scale_options**.

`yrange(numlist)` numlist of size 2 to set the y range; see [R] **scale_options**.

The `disco_weight` command

The `disco_weight` command displays and stores synthetic control weights after `disco` estimation, matching numeric unit IDs (*id_var*) with their corresponding string names (*name_var*). The results are stored in a new frame for further analysis or easy export to tables. The command requires that `disco` has been run previously.

Syntax `disco_weight` *id_var name_var* [, *options*]

Options The options for `disco_weight` are the following.

`n(#)` specifies the number of top weights to display and store. The default is 5.

`format(string)` specifies the display format for weights. The default is `%12.4f`.

`frame(string)` specifies the name of the frame where results will be stored. The default is `"disco_weights"`. If a frame with this name already exists, it will be replaced.

`round(#)` specifies the rounding precision for weights. The default is 0.0001.

Stored results `disco_weight` stores the following in `r()`:

Scalars		
	`r(n)`	number of top weights displayed
Macros		
	`r(cmd)`	`disco_weight`
	`r(format)`	display format used
	`r(frame)`	name of frame where results are stored

4 Empirical Illustration: Van Dijcke et al. (2026)

Van Dijcke et al. (2026) study the effect of return-to-office (RTO) mandates on firms' workforce composition. The question that motivates the paper is whether a firm-wide mandate to work from the office at least one day a week causes junior and senior employees to depart the firm at different rates. This is a setting where the treatment (RTO mandate) affects an entire group (a firm) at once, and the outcome of interest is inherently a distribution, in this case the distribution of employee tenure and titles. The paper estimates a distributional synthetic control for three large tech firms that

were early to introduce firm-wide RTO mandates, Microsoft, SpaceX, and Apple. The paper's main finding is that the RTO mandates at these large technology firms were followed by a significant decrease in employee tenure at the top tail of the distribution, with employees at the upper deciles departing Microsoft approximately 2 months earlier due to the RTO mandate; and a shift towards more junior titles, with an increase in employees at or below the manager level of around 4%. We reproduce those results below.

The do-files and (perturbed) data that reproduce this section are distributed with this article.

4.1 Data and Methods

Data Overview. In the paper, we use quarterly résumé data from People Data Labs,[2] a large provider of individual-level employment records that covers over half of the total headcount for most major tech firms (see Van Dijcke et al. 2026 for details). Each firm-quarter includes a repeated cross-section of employees, from which we observe job tenure and a seniority rank (ranging from "unpaid" to "CXO"). We merge in information on return-to-office (RTO) mandates gathered from public announcements, employee forums, and the Flex Index by Scoop Technologies, as well as layoff records from layoffs.fyi. Our main estimation sample focuses on the return to office at Microsoft in April 2022, plus a control pool of firms never adopting RTO within the observation window. For more details on the data, see Van Dijcke et al. (2026). In practice, we use a perturbed version of the original data for confidentiality reasons, where we add uniform noise to the outcome variables. This preserves the shape of the distributions while keeping the dataset shareable.

Empirical Approach. Using these data, we then apply a distributional synthetic controls design that computes synthetic control weights to closely match Microsoft's distribution of employee tenure in the two quarters before its RTO. The post-RTO gap between the observed and synthetic distributions isolates the policy's causal effect on workforce composition.

In the notation from Section 2, i refers to a single employee at firm j (with $j = 1$ corresponding to Microsoft), t refers to a quarter of the year, and Y_{ijt} is one of two main outcomes:

1. $Tenure_{ijt}$, the number of days employee i has been employed at company j.
2. $Title_{ijt}$, an integer classification of employee i's job title at firm j, ranging from 1 (trainee) to 10 (C-suite). See Van Dijcke et al. (2026, Table 1) for the full classification.

We solve (3) for $Tenure_{ijt}$ (as it is continuous) and (5) for the categorical $Title_{ijt}$. Put differently, it makes sense to interpolate over the support of tenure (e.g., 5 days, 5.5 days), but it generally does not make sense to talk about an employee being "25% junior

2. https://www.peopledatalabs.com/

associate and 75% manager."

4.2 Results: Quantile Approach

First, we reproduce the main Figure 4 in the paper. To that end, we start by loading and inspecting the (perturbed) data:

```
. use "tenure_anonymized.dta", clear
. list in 1/5, ab(20)
```

	time_col	id_col	company_name	y_col
1.	2	17	oracle	1682.25
2.	3	1	deloitte	375.783
3.	2	72	3m	5276.48
4.	3	2	microsoft	957.55
5.	2	2	microsoft	2745.96

The data structure shows the essential columns required for estimation: (1) a *time column*, indicating the time period (quarters of 2022); (2) a numeric ID column indicating the ID of the aggregate unit (firms, e.g. ID "17" for Oracle); (3) a numeric outcome variable (days the employee at Oracle had been working there at the start of quarter 2). There is also an optional string column indicating the name of the aggregate unit ("Oracle"), which can be used in the `disco_weight` command to inspect synthetic control weights by name.

With these data, we can replicate the figure with:

```
. disco y_col id_col time_col, idtarget(2) t0(3) agg("quantileDiff") ///
>         seed(12143) g(10) m(100) ci boots(300)
```

Here, we evaluate the quantile functions on 10 grid points using `g(10)`, but use 100 points for estimation (`m(100)`) to approximate the integral in (4). In the paper, we used 1,000 samples; we lower that to 100 here for replication speed. Similarly, we compute confidence intervals via the `ci` option with only 300 bootstrap repetitions (`boots(300)`) instead of 1,000 as in the paper.

After estimation, we first examine the top 5 weights by company name:

```
. disco_weight id_col company_name, n(5)
Top 5 weights:
```

name	weight
amazon	.2203
autodesk	.1271
cisco	.1066
dell technologies	.0991
slalom consulting	.0962

These weights are very close to those in Van Dijcke et al. (2026, Table 3), despite the data perturbation. As noted there, firms like Amazon, Cisco, and Dell receive high weights because they are highly similar to Microsoft (the treated firm).

We then directly plot the quantile effects, defined as the differences between the treated and synthetic quantile functions at each of the 10 deciles:

```
. disco_plot, title(" ") ytitle("Difference in Tenure (Days)") ///
>      hline(0) scheme("stsj")
```

This command creates Figure 2, which closely resembles Figure 4 in the paper.

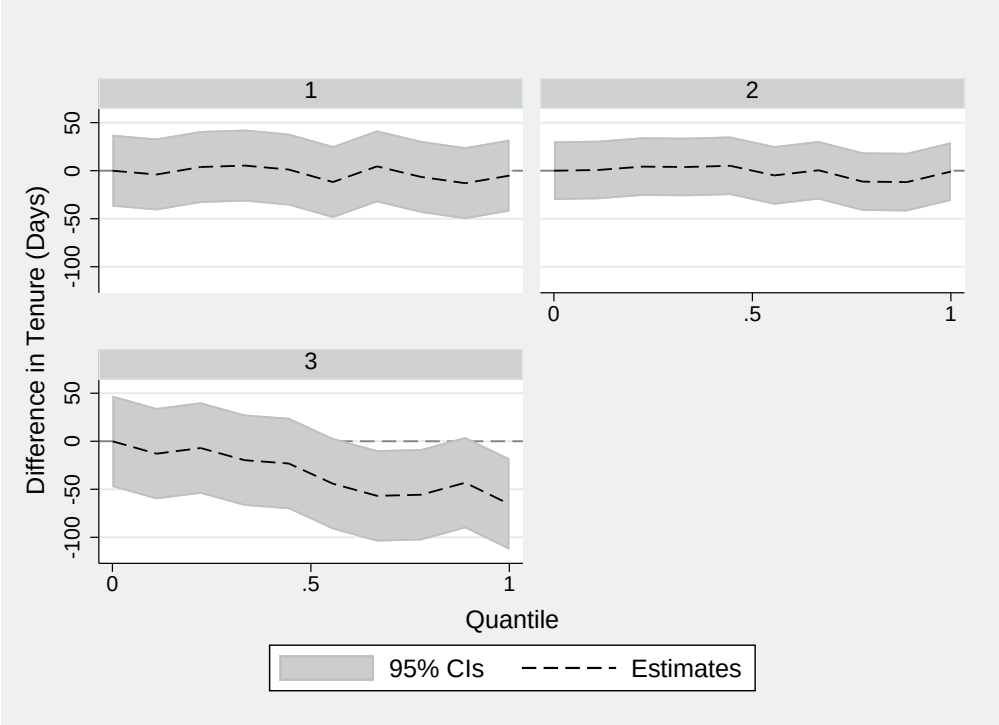


Figure 2: Replication of Tenure Result

As discussed in the paper, panel 3 in the figure suggests a statistically significant decrease in tenure (about 50 days) for employees in the top 2–3 deciles of the tenure distribution, but no such decrease at the lower deciles. In other words, the RTO caused longer-tenured employees to leave Microsoft, but not those with shorter tenures. Because these effects are calculated as the difference between the observed and the synthetic quantile function, they have a counterfactual interpretation — i.e., the drop in tenure at the top deciles is relative to a "synthetic" Microsoft that did not return to the office. Finally, panels 1 and 2 in the figure show the quantile differences for the two quarters before the RTO. Since 0 is included in the 95% confidence bands there, we see that the

synthetic control replicated Microsoft's tenure distribution well, supporting its use for constructing post-treatment counterfactuals.

Although not shown in the paper, a researcher may be interested in comparing the synthetic and observed quantile functions side by side. This can be done with:

```
. disco_plot, title(" ") ytitle("Tenure (Days)") agg("quantile")
```

which yields Figure 3.

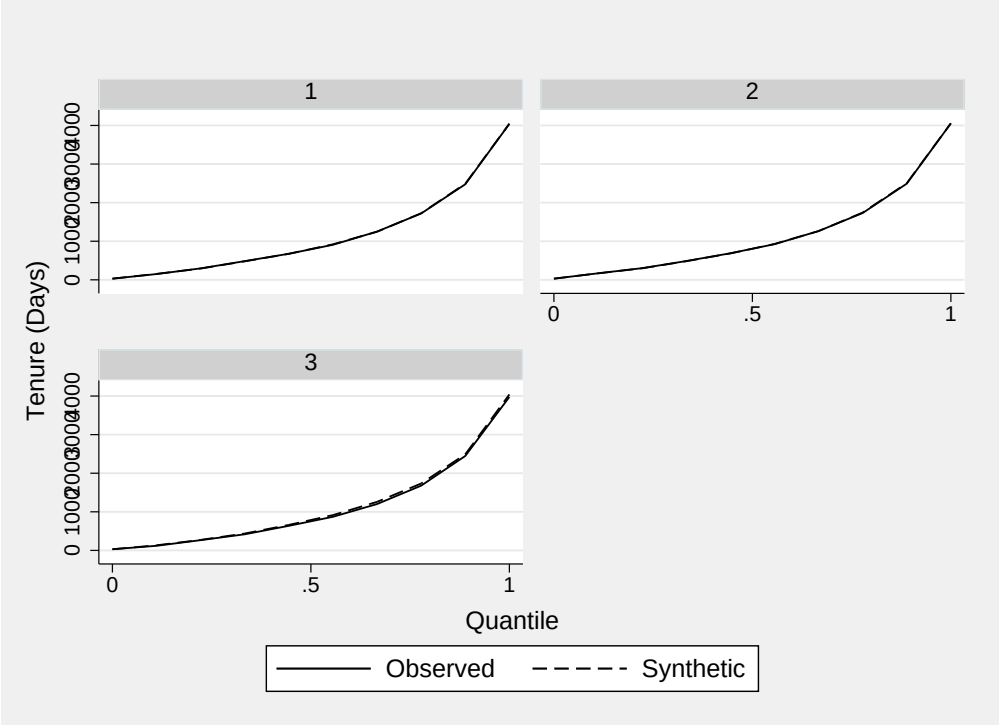


Figure 3: Replication of Tenure Result

We observe similar patterns here: a strong pre-treatment fit, with the two quantile functions overlapping, and a downward shift in the synthetic quantile function at the top deciles after the RTO. Additionally, it clarifies the absolute magnitudes in the distribution: the top 3 deciles, where the largest effects occur, correspond to employees who have been at Microsoft for at least 1,500 days (about 4 years).

Finally, a researcher may want to study the treatment effects in aggregate parts of the distribution. To do so, we can run:

```
. disco_estat summary
Summary of quantile effects
```

Period	Range	Effect	Std. Err.	[95% Conf. Interval]

```
------------------------------------------------------------------------------
        3  0.00-0.25          -6.659      24.256     -54.201      40.883
        3  0.25-0.50         -21.443      24.256     -68.985      26.099
        3  0.50-0.75         -50.509      24.256     -98.052      -2.967*
        3  0.75-1.00         -54.751      24.256    -102.293      -7.209*
------------------------------------------------------------------------------
* denotes significance at the 95% confidence level
```

By default, the command aggregates the treatment effects into four buckets corresponding to the four quartiles. We see statistically significant effects in the top two quartiles, corresponding to decreases of about 51 and 55 days in employee tenure; the effect in the third quartile is borderline, with the confidence interval barely excluding zero.

4.3 Results: CDF Approach

An analogous exercise can be carried out for the categorical *Title* variable, except that we now want the command to solve (5), as this variable takes integer values from 1 to 10. Hence, we specify the `mixture` option and force the command to use a grid of 10 points by setting `g(10)` and `m(10)`. Concretely:

```
. use "titles_anonymized.dta", clear
. list in 1/5, ab(20)

     +-------------------------------------------------+
     | time_col   id_col   company_name   y_col |
     |-------------------------------------------------|
  1. |        3       17         oracle       8 |
  2. |        2        1       deloitte       5 |
  3. |        3        1       deloitte       6 |
  4. |        1        1       deloitte       5 |
  5. |        2      245         splunk       4 |
     +-------------------------------------------------+

.
. disco y_col id_col time_col, idtarget(2) t0(3) agg("cdfDiff") seed(12143) ///
>         mixture g(10) m(10) ci boots(300)
. disco_plot, title(" ") ytitle("Change in CDF") hline(0) categorical ///
>         scheme("stsj") color("bluishgray")
```

Since `y_col` is clearly categorical, specifying `categorical` in the `disco_plot` command generates a bar plot rather than a line plot. The resulting figure, shown in Figure 4, closely resembles Figure 5 in Van Dijcke et al. (2026), though the data perturbation slightly widens the confidence intervals.

This figure can be interpreted as showing a statistically significant increase in the mass under the CDF for employees with titles below the (senior) manager level (4), but no such increase for higher-ranked employees. By the nature of cumulative distribution functions, this means Microsoft's workforce rebalanced toward lower-ranked employees after the RTO, likely due to an outflow of senior talent.

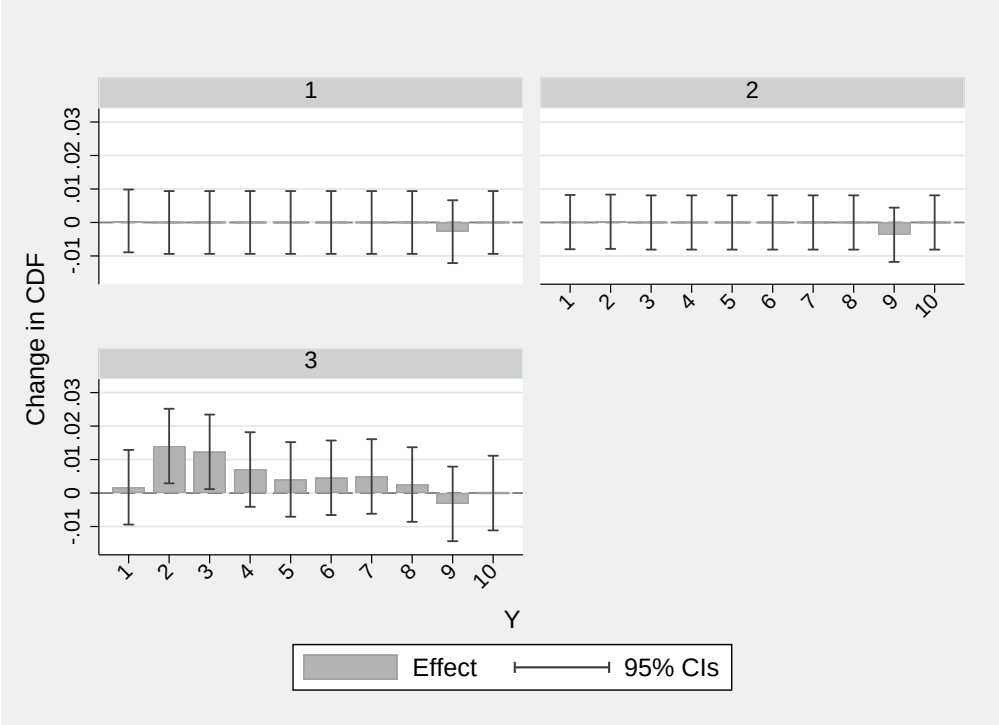


Figure 4: Replication of Titles Result

5 Conclusions

We have presented `disco`, a Stata implementation of the Distributional Synthetic Controls estimator of Gunsilius (2023), and show how it can be used to reproduce the distributional effects of RTO mandates in Van Dijcke et al. (2026).

Two extensions would make the command more useful. The first is a multivariate version that matches the joint distribution of several outcomes at once instead of one at a time, following Gunsilius et al. (2024). That turns the weight problem into a multidimensional optimal-transport problem, which is harder to solve and will need a dedicated routine. The second is faster inference. The bootstrap still drives the runtime: the command already sorts and caches the outcome data across replications, but spreading the bootstrap loop over multiple cores would cut the remaining time by a further large factor.

6 References

Abadie, A., A. Diamond, and J. Hainmueller. 2010. Synthetic control methods for comparative case studies: Estimating the effect of California's tobacco control program.

Journal of the American Statistical Association 105: 493–505.

Abadie, A., and J. Gardeazabal. 2003. The economic costs of conflict: A case study of the Basque Country. *American Economic Review* 93: 113–132.

Agueh, M., and G. Carlier. 2011. Barycenters in the Wasserstein space. *SIAM Journal on Mathematical Analysis* 43: 904–924.

Chen, Y.-T. 2020. A distributional synthetic control method for policy evaluation. *Journal of Applied Econometrics* 35: 505–525.

Chetverikov, D., B. Larsen, and C. Palmer. 2016. IV quantile regression for group-level treatments, with an application to the distributional effects of trade. *Econometrica* 84: 809–833.

Di Gaspero, L. 2016. Quadprog++: a C++ library for Quadratic Programming. https://github.com/liuq/QuadProgpp.

Doudchenko, N., and G. W. Imbens. 2016. Balancing, regression, difference-in-differences and synthetic control methods: A synthesis. Technical report, National Bureau of Economic Research.

Goldfarb, D., and A. Idnani. 1983. A numerically stable dual method for solving strictly convex quadratic programs. *Mathematical Programming* 27: 1–33.

Gunsilius, F., M. H. Hsieh, and M. J. Lee. 2024. Tangential wasserstein projections. *Journal of Machine Learning Research* 25(69): 1–41.

Gunsilius, F. F. 2023. Distributional synthetic controls. *Econometrica* 91: 1105–1117.

Hausman, J. A., and W. E. Taylor. 1981. Panel data and unobservable individual effects. *Econometrica* 1377–1398.

Van Dijcke, D. 2025. Regression discontinuity design with distribution-valued outcomes. *arXiv preprint arXiv:2504.03992* .

Van Dijcke, D., F. Gunsilius, and S. He. 2024. *DiSCos: Distributional Synthetic Controls Estimation*. http://www.davidvandijcke.com/DiSCos.

Van Dijcke, D., F. Gunsilius, and A. Wright. 2026. Return to Office and the Tenure Distribution. *Review of Economics & Statistics* Conditionally Accepted.

Xu, Y. 2017. Generalized synthetic control method: Causal inference with interactive fixed effects models. *Political Analysis* 25: 57–76.

About the authors

Florian Gunsilius is an Associate Professor in the Department of Economics at Emory University. His research interests are nonparametric approaches for statistical identification, estimation, and inference. His current focus is on statistical optimal transport theory, mean field estimation, causal inference, and free discontinuity problems.

David Van Dijcke is an Assistant Professor in the Department of Economics at the University of Virginia. His research interests focus on causal inference with non-Euclidean and distributional data, nonparametric estimation, and regression discontinuity designs.